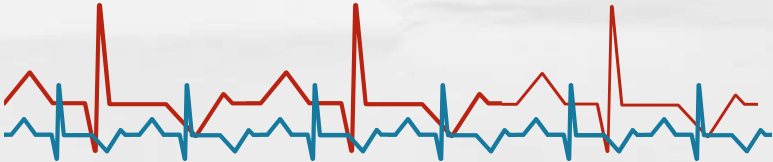
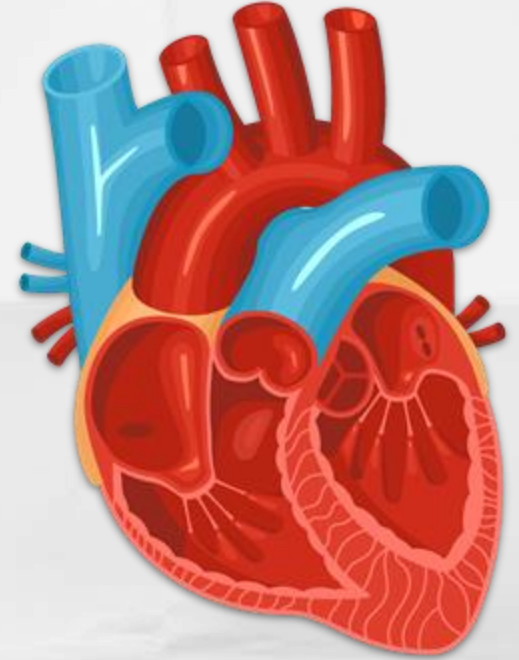


HOSPITAL MANAGEMENT SYSTEM

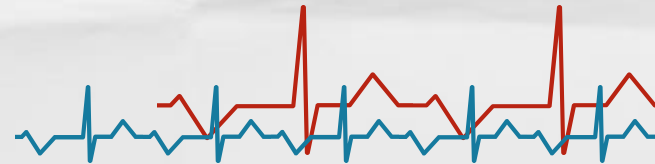
Cardiology Department



INTRODUCTION



This project represents a specialized Cardiology Management System within a broader Hospital Management framework. It involves designing a structured database using ER modeling, normalization, and SQL to efficiently manage heart-related patient care and administrative operations. Focusing on cardiology helps narrow the scope and streamline system development.

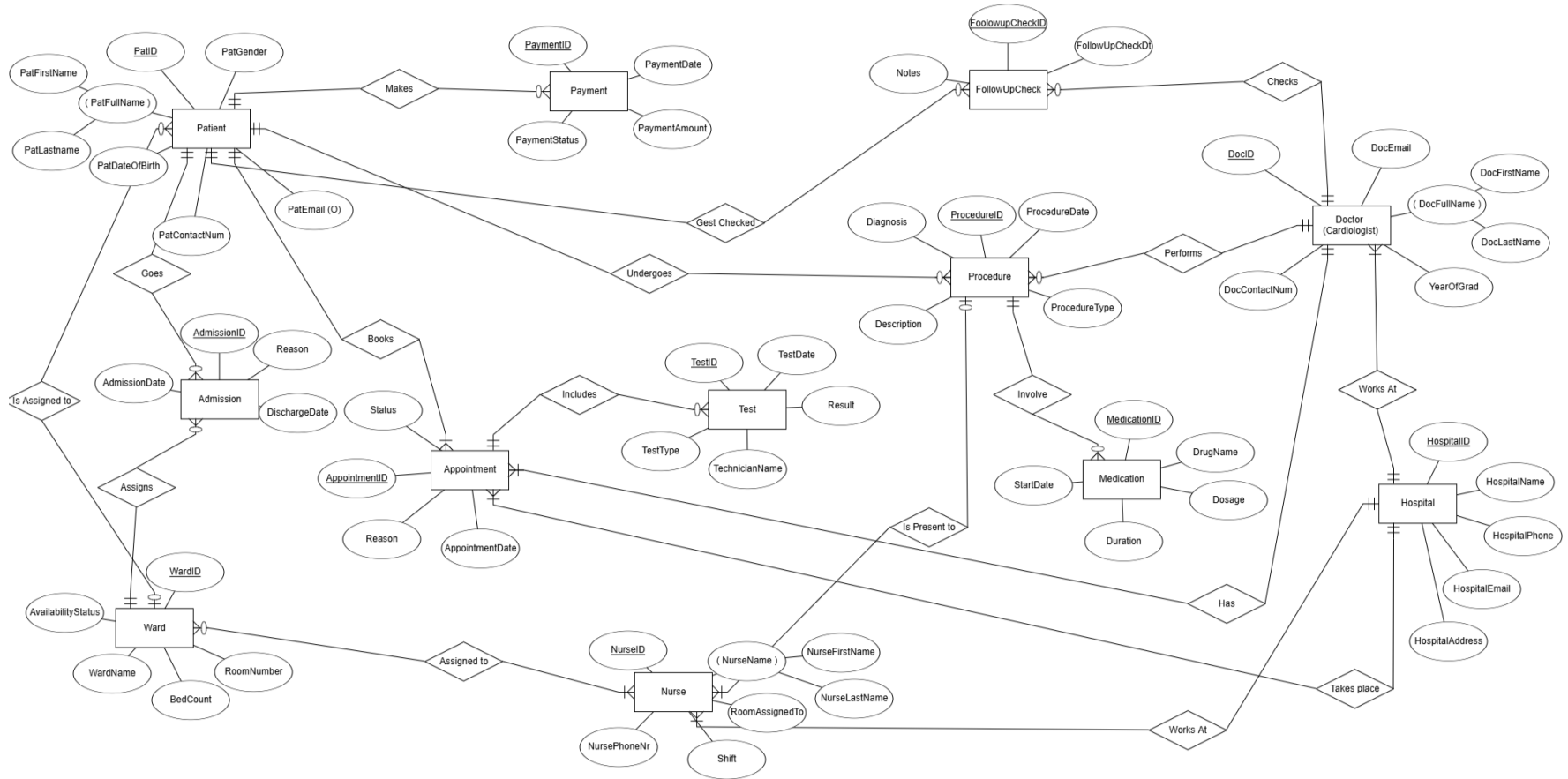


OBJECTIVES:

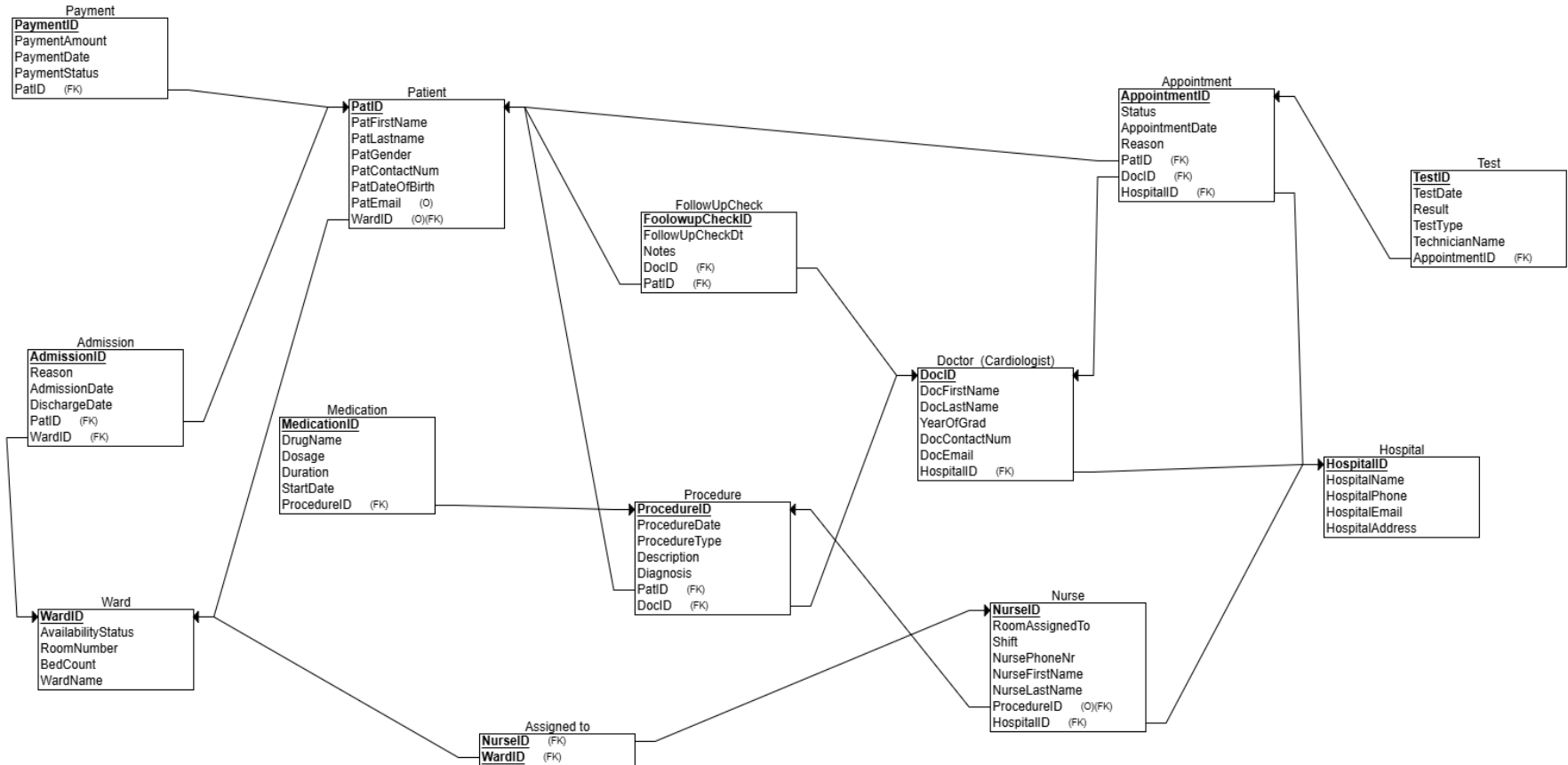
- To track and manage patients, cardiologists (doctors), appointments, diagnostic tests, treatments, medications, procedures, admissions, payments, wards, and nurses.
- To improve efficiency, traceability, and accuracy in handling cardiology-specific workflows.
- To ensure data normalization, integrity, and relational structure for effective querying and reporting.



ER Diagram



Relational Schema



CREATION OF TABLES

Patient

<u>PatID</u>	PatFirstName	PatLastName	PatGender	PatContactNum	PatDtOfBirth	PatEmail	WardID
01	Emily	Johnson	F	1467800325	2000-10-26	emily@gmail.com	1
02	John	Williams	M	3689022002	1993-05-29	john@gmail.com	2
03	Tom	Wilson	M	0975541903	1985-06-21	tom@gmail.com	3
04	Jane	White	F	1579301957	1997-11-07	jane@gmail.com	
05	Jackson	Roberts	M	7893567723	1966-07-29	jackson@gmail.com	
06	Olivia	Scott	F	9088845230	1973-11-27	olivia@gmail.com	5



Hospital

<u>HospitalID</u>	HospitalName	HospitalPhone	HospitalEmail	HospitalAdd
1	Cardio Health Center	555-1234	info@cardiohc.org	123 Heart Lane

Doctor

<u>DocID</u>	DocFirstName	DocLastName	YearOfGrad	D.ContactNum	D.Email	HospitalID
AA	Emma	Yang	2008	2000000000	emma1@hospital.org	1
AB	Gregory	Bailey	2010	2000000001	gregory1@hospital.org	1
BB	Alex	James	1993	2000000002	alex1@hospital.org	1
BC	Cristina	Grey	2015	2000000003	cristina1@hospital.org	1



Follow-up-check

<u>FollowupcheckID</u>	FollowupcheckDate	Notes	DocID	PatID
1	2025-02-14	Patient improving	AA	01
2	2025-04-09	Blood pressure stable	BB	03
3	2025-05-02	Further test advised	BC	05
4	2025-04-21	No complications	AA	02
5	2025-05-03	ECG clear,no arrhythmia	AA	02
6	2025-02-12	MRI next week	BB	03



Payment

<u>PaymentID</u>	Payment Amount	PaymentDate	Payment Status	PatID
0	350.75	2025-03-25	Paid	02
1	500.00	2025-01-03	Paid	03
2	450.00	2025-02-22	Pending	03
3	530.20	2024-09-01	Declined	04
4	475.00	2025-02-16	Pending	06
5	200.00	2024-10-11	Paid	01
6	370.00	2025-01-24	Paid	03
7	190.00	2025-04-14	Pending	05
8	250.00	2025-03-27	Paid	02



Ward

<u>WardID</u>	AvailabilitySt.	RoomNumber	BedCount	WardName
1	Occupied	101	3	Ward-1
2	Available	102	4	Ward-2
3	Available	103	2	Ward-3
4	Occupied	104	5	Ward-4
5	Occupied	105	2	Ward-5
6	Available	106	6	Ward-6

Assigned to

<u>NurseID</u>	<u>WardID</u>
N1	3
N2	6
N3	2
N4	1
N5	3

Nurse

<u>NurseID</u>	N.FirstName	N.LastName	RoomAssigned to	N.PhoneNr	Shift	Procedure ID	HospitalID
N1	Daniel	Brown	101A	3000000000	Day	P3	1
N2	Anna	Swift	101A	3000000001	Night	P6	1
N3	Eva	Clark	102B	3000000002	Night	P4	1
N4	Sue	Johnson	103C	3000000003	Day	P1	1
N5	Tina	Smith	102B	3000000004	Day	P2	1



Procedure

<u>ProcedureID</u>	ProcedureDate	ProcedureType	Description	Diagnosis	PatID	DocID
P1	2024-11-20	Coronary Angioplasty	Widening narrowed heart arteries	Coronary Artery Disease	03	AA
P2	2024-10-05	Pacemaker insertion	Implant for arrhythmia control	Bradycardia	01	BC
P3	2025-03-11	Echocardiography	Imaging for valve assesment	Valve Disease	02	BB
P4	2024-07-15	Cardioversion	Restoring normal rhythm	Atrial Fibrillation	04	BC
P5	2025-04-27	Cardiac Catheterization	Diagnostic catheterization	Chest pain	05	AB
P6	2025-02-08	Holter Monitoring	24-hour ECG monitoring	Palpitations	06	AA



Medication

<u>MedicationID</u>	DrugName	Dosage	DurationDays	StartDate	ProcedureID
M0	Aspirin	75 mg	14	2024-11-23	P1
M1	Isoprotenerol	10 mg	12	2024-10-08	P2
M2	Apixaban	15 mg	15	2024-07-18	P4
M3	Nitroglycerin	20 mg	7	2025-04-30	P5
M4	Propranorol	40 mg	10	2025-02-10	P6



Appointment

<u>AppointmentID</u>	Status	Reason	AppointmentDate	PatID	DocID	HospitalID
A000	Scheduled	Routine check	2024-09-20	01	AA	1
A001	Scheduled	Follow-up	2025-01-17	03	AB	1
A010	Completed	Initial Consult	2025-01-05	03	AB	1
A011	Cancelled	Follow-up	2025-05-01	02	BC	1
A100	Completed	Routine check	2024-10-25	04	BB	1
A101	Completed	Routine check	2025-03-13	05	AB	1
A110	Scheduled	Initial consult	2025-04-19	06	BB	1
A111	Completed	Routine check	2025-02-16	06	AA	1



Test

<u>TestID</u>	TestDate	Result	TestType	TechnicianName	AppointmentID
T1	2025-02-01	AbNormal	ECG	Tom Hardy	A001
T2	2024-09-29	Normal	Echocardiogram	Sara Finn	A000
T3	2024-11-01	Abnormal	Stress Test	Sara Finn	A100
T4	2025-03-16	Abnormal	Cardiac MRI	Liam Brooks	A101
T5	2025-04-23	Normal	ECG	Tom Hardy	A110
T6	2025-02-20	Normal	Chest X-ray	Sara Finn	A111



Admission

<u>AdmissionID</u>	Reason	AdmissionDate	Discharge Date	PatID	WardID
1	Irregular heartbeat	2024-08-17	2024-11-22	01	1
2	Chest pain	2024-04-13	2024-06-24	05	2
3	Palpitations	2025-03-08	2025-05-11	06	2
4	Uncontrolled hypertension	2025-04-01	2025-05-14	02	4
5	Irregular heartbeat	2024-09-21	2024-12-19	03	5
6	Abnormal ECG findings	2025-02-16	2025-04-28	04	3



Database Improvement-Normalization

We focused on **normalization** to enhance the structure and performance of our database system. It applies a series of *normal forms* (1NF, 2NF, 3NF), each with progressively more refined rules for organizing data.

1NF

Each table must have unique rows, and every column must contain only single, atomic values—no multiple or repeating values allowed.

2NF

For a relational table, the process of normalization starts by examining if a relational table is in 2NF. A table is in 2NF if it is in 1NF and if it does not contain partial functional dependencies.

3NF

A table is in 3NF if it is in 2NF and if it does not contain transitive functional key dependencies.



Why this matters?

- 1.Faster performance:** Well-structured tables reduce data repetition, leading to quicker query execution and optimized storage.
- 2.Improved consistency:** By minimizing redundancy, normalization ensures that each piece of information is stored in one place, reducing the risk of conflicting data.
- 3.Easier maintenance:** With clear relationships and cleaner design, it's simpler to update, extend, or troubleshoot the database as the system grows.

1)Payment table

<u>PaymentID</u>	PaymentAmount	PaymentDate	PaymentStatus	PatID
------------------	---------------	-------------	---------------	-------

1NF-> Unique rows , no repeated values . Already in 1NF form

2NF ->No Partial key dependencies . Already in that form

3NF->No Transitive key dependencies . Already in that form

2)Admission table

<u>AdmissionID</u>	Reason	Admissiondate	DischargeDate	PatID	WardID
--------------------	--------	---------------	---------------	-------	--------

1NF->Unique rows, no repeated values. Already in that form

2NF-> No PKD . Already in that form .

3NF-> No TKD . Already in that form .

3)Ward table

<u>WardID</u>	AvailabilTyStatus	RoomNum	BedCount	WardName
---------------	-------------------	---------	----------	----------

1NF-> Unique Rows. Already in that form

2NF->No partial Key Dependency

1)

<u>WardID</u>	WardName
---------------	----------

FKD

2)

<u>WardID</u>	AvailabilityStatus	RoomNum	BedCount
---------------	--------------------	---------	----------

FKD

3NF-> No TKD . Already in that form

4) Medication Table

<u>MedicationID</u>	DrugName	Dosage	Duration	StartDate	Procedure
---------------------	----------	--------	----------	-----------	-----------

1NF-> Unique rows . Already in that form.

2NF->No PKD

<u>MedicationID</u>	DrugName
---------------------	----------

FKD

<u>MedicationID</u>	Dosage	Duration	StartDate	Procedure
---------------------	--------	----------	-----------	-----------

FKD

3NF-> No TKD . Already in that form

5)Patient table

<u>PatientID</u>	PatFName	PatLName	Gender	Contact	DateofBirth	WardID
------------------	----------	----------	--------	---------	-------------	--------

1NF-> Unique row . No repeated values .Already in that form

2NF->No PKD

1)

<u>PatID</u>	PatFName	PatLName
--------------	----------	----------

FKD

2)

<u>PatID</u>	PatGender	PatContactNum	PatDateOfBirth	PatEmail	WardID
--------------	-----------	---------------	----------------	----------	--------

FKD

6)FollowUpCheck table

FollowUpCheckID	Date	Notes	DoctorID	PatientID
-----------------	------	-------	----------	-----------

FKD

1NF-> Unique values . Already in that form

2NF->No PKD. Already in that form

3NF-> No TKD . Already in that form

7)Procedure table

<u>ProcedureID</u>	ProcedureDate	ProcedureType	Description	Diagnosis	PatID	DocID
--------------------	---------------	---------------	-------------	-----------	-------	-------

FKD

1NF->Unique rows. Already in that form

2NF->No PKD. Already in that form

3NF->No TKD. Already in that form

8)Nurse table

<u>NurseID</u>	NurseFName	NurseLName	PhoneNum	RoomAssignedTo	Shift	ProcedureID	HospitalID
----------------	------------	------------	----------	----------------	-------	-------------	------------

1NF-> Unique values

2NF-> No PKD

1)

<u>NurseID</u>	NurseFName	NurseLName
----------------	------------	------------

FKD

2)

<u>NurseID</u>	PhoneNum	RoomAssignedTo	Shift	ProcedureID	HospitalID
----------------	----------	----------------	-------	-------------	------------

FKD

9)Assigned to table

<u>NurseID</u>	WardID
----------------	--------

FKD

1ND->Unique values . Already in that form

2NF->No PKD . Already in that form

3NF-> No TKD. Already in that form

10) Doctor Table

<u>DocID</u>	DocFName	DocLName	YearOf Grad	ContactNum	Email	HospitalID
--------------	----------	----------	----------------	------------	-------	------------

1NF-> Unique Values . Already in that form

2NF->No PKD

1)

<u>DocID</u>	DocFName	DocLName
--------------	----------	----------

FKD

2)

<u>DoctorID</u>	YearOFGrad	Contact	Email	HospitalID
-----------------	------------	---------	-------	------------

FKD

3NF->No TKD. Already in that form

11)Appointment table

<u>AppointmentID</u>	Status	AppointmentDate	Reason	PatID	DocID	HospitalID
----------------------	--------	-----------------	--------	-------	-------	------------

FKD

1NF->Unique Rows.Already in taht state

2NF-> No PKD. Already in that state

3NF-> No TKD. Already in that state

12)Hospital table

<u>HospitalID</u>	HospitalName	HospitalPhoneNum	HospitalEmail	HospitalAdress
-------------------	--------------	------------------	---------------	----------------

1NF-> Unique values. Already in that form

2NF-> No PKD .

1)

<u>HospitalID</u>	HospitalName
-------------------	--------------

FKD

2)

<u>HospitalID</u>	HospitalPHoneNum	HospitalEmail	HospitalAdress
-------------------	------------------	---------------	----------------

FKD

3NF->Nn TKD . Already in that form

13)Test table

<u>TestID</u>	TestDate	Result	TestType	TechName	AppointmentID
---------------	----------	--------	----------	----------	---------------

1NF->Unique values. Already in that form

2NF->

1)

<u>TestID</u>	TestType
---------------	----------

FKD

2)

<u>TestID</u>	TestDate	Result	TechType	AppointmentID
---------------	----------	--------	----------	---------------

FKD

3NF-> No TKD . Already in that form

SQL Code Implementation

CREATE & INSERT DATA

CREATE TABLE patient

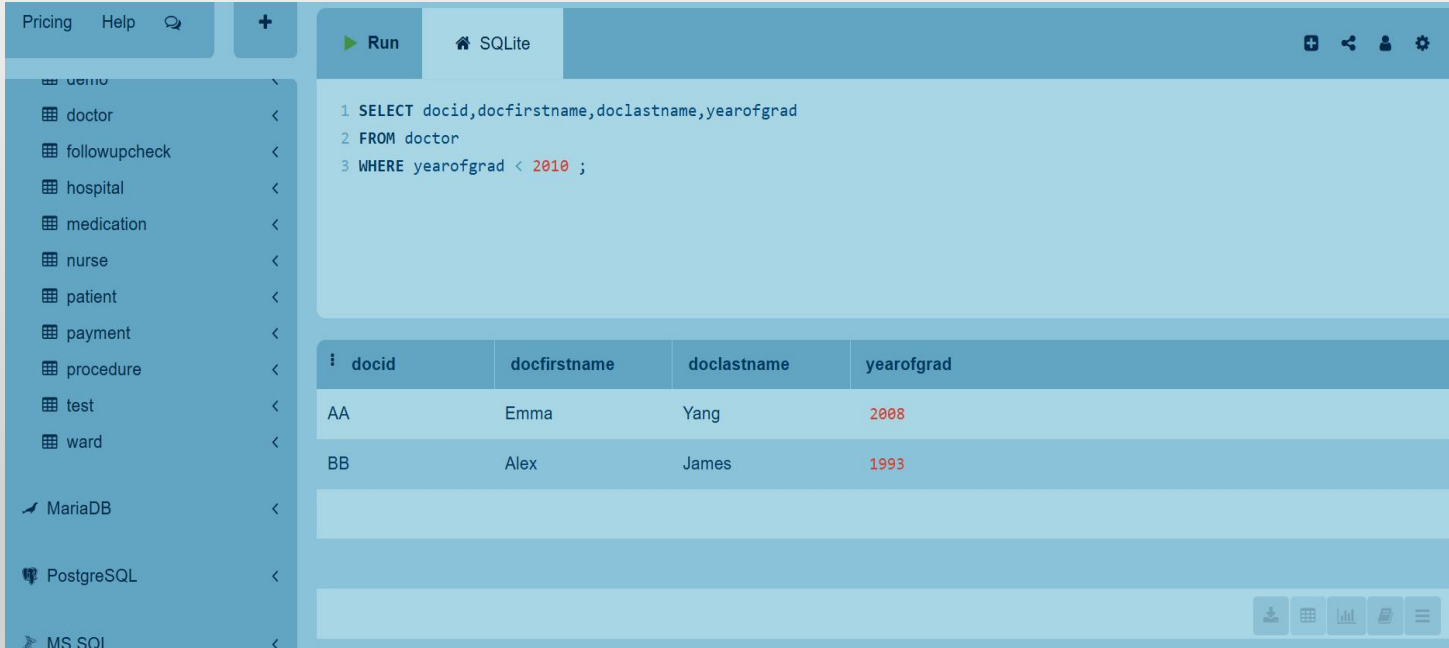
```
( patid CHAR(2) NOT NULL,  
  patfirstname VARCHAR(12) NOT NULL,  
  patlastname VARCHAR(12) NOT NULL,  
  patgender CHAR(1) NOT NULL,  
  patcontactnum VARCHAR(10) NOT NULL,  
  patdtofbirth DATE NOT NULL,  
  patemail VARCHAR(20) NOT NULL,  
  wardid CHAR(1),  
  PRIMARY KEY (patid),  
  FOREIGN KEY (wardid) REFERENCES ward(wardid) );
```

```
INSERT INTO patient VALUES ('01','Emily','Johnson','F','1467800325','2000-10-  
26','emily@gmail.com','1');  
INSERT INTO patient VALUES ('02','John','Williams','M','3689022002','1993-05-  
29','john@gmail.com','2');  
INSERT INTO patient VALUES ('03','Tom','Wilson','M','0975541903','1985-06-  
21','tom@gmail.com','3');  
INSERT INTO patient VALUES ('04','Jane','White','F','1579301957','1997-11-  
07','jane@gmail.com',NULL);  
INSERT INTO patient VALUES ('05','Jackson','Roberts','M','7893567723','1966-07-  
29','jackson@gmail.com',NULL);  
INSERT INTO patient VALUES ('06','Olivia','Scott','F','9088845230','1973-11-  
27','olivia@gmail.com','5');
```

Full SQL code is provided in the project documentation.

Managerial Queries

1.Retrieve the doc id,doc firstname,doc lastname,year of graduation for the doctors who graduated before the year 2010



The screenshot shows a database management interface. On the left is a sidebar with a list of database tables: doctor, followupcheck, hospital, medication, nurse, patient, payment, procedure, test, ward, MariaDB, PostgreSQL, and MS SQL. The main area is divided into two sections. The top section contains a SQL query:

```
1 SELECT docid,docfirstname,docl lastname,yearofgrad  
2 FROM doctor  
3 WHERE yearofgrad < 2010 ;
```

 The bottom section displays the results of the query in a table format. The table has four columns: docid, docfirstname, doclastname, and yearofgrad. The results are as follows:

docid	docfirstname	doclastname	yearofgrad
AA	Emma	Yang	2008
BB	Alex	James	1993

2.Retrieve pat id,pat firstname,pat lastname,ward id to list all patients with their assigned ward

The screenshot shows a database management interface. On the left is a sidebar with a tree view containing various database tables: doctor, followupcheck, hospital, medication, nurse, patient, payment, procedure, test, ward, MariaDB, PostgreSQL, and MS SQL. The main area is divided into two sections. The top section contains a SQL query editor with the following code:

```
1 SELECT patid,patfirstname,patlastname,wardid
2 FROM patient
3 WHERE wardid != 'NULL' ;
4
```

The bottom section displays the query results in a table format. The table has four columns: patid, patfirstname, patlastname, and wardid. The results are as follows:

patid	patfirstname	patlastname	wardid
01	Emily	Johnson	1
02	John	Williams	2
03	Tom	Wilson	3
06	Olivia	Scott	5

At the bottom right of the results area, there are icons for downloading, viewing as a table, chart, document, and menu.

3.Retrieve paymentid,payment amount,payment status,pat id for each patient with ID 03,sorted by payment amount

Pricing Help

Create a database linked to your account.

This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

- admission
- appointment
- assignedto
- demo
- doctor
- followupcheck

Run SQLite

```
1 SELECT paymentid,paymentamount,paymentstatus,patid
2 FROM payment
3 WHERE patid = '03'
4 ORDER BY paymentamount ;
5
6
7
```

paymentid	paymentamount	paymentstatus	patid
6	370	Paid	03
2	450	Pending	03
1	500	Paid	03

4.Retrieve the record for each appointment whose status contains the phrase 'Completed'

Pricing Help

Create a database linked to your account.

This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

- admission
- appointment
- assignedto
- demo
- doctor
- followupcheck
- hospital

Run SQLite

```
1 SELECT *
2 FROM appointment
3 WHERE status LIKE '%Completed%';
4
5
6
7
8
```

app...	status	reason	appoin...	patid	docid	hospitalid
A010	Completed	Initial co...	2025-01...	03	AB	1
A100	Completed	Routine ...	2024-10...	04	BB	1
A101	Completed	Routine ...	2025-03...	05	AB	1
A111	Completed	Routine ...	2025-02...	06	AA	1

Download Table View Chart Print Menu

5.Retrieve the total number of beds in available wards

Pricing Help

Create a database linked to your account.

This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

- admission
- appointment
- assignedto
- demo
- doctor
- followupcheck

Run SQLite

```
1 SELECT availabilityst,SUM(bedcount) AS totalavailablebeds
2 FROM ward
3 WHERE availabilityst = 'Available' ;
4
5
6
7
8
```

availabilityst	totalavailablebeds
Available	12

Download Table View Chart Print Menu

6. For each doctor, retrieve the doc id, number of appointments supplied by the doctor

The screenshot displays a web application interface for managing a SQLite database. The top navigation bar includes links for Pricing, Help, and a plus icon. The main content area is divided into a left sidebar and a central workspace.

Left Sidebar:

- Top section: "Create a database linked to your account. This is only available with a paid subscription."
- Database section: "SQLite" with a dropdown arrow.
- Database details: "0.1.4 beta (Mem..."
- Table list: A list of tables with expand/collapse icons: admission, appointment, assignedto, demo, doctor, followupcheck, and hospital.

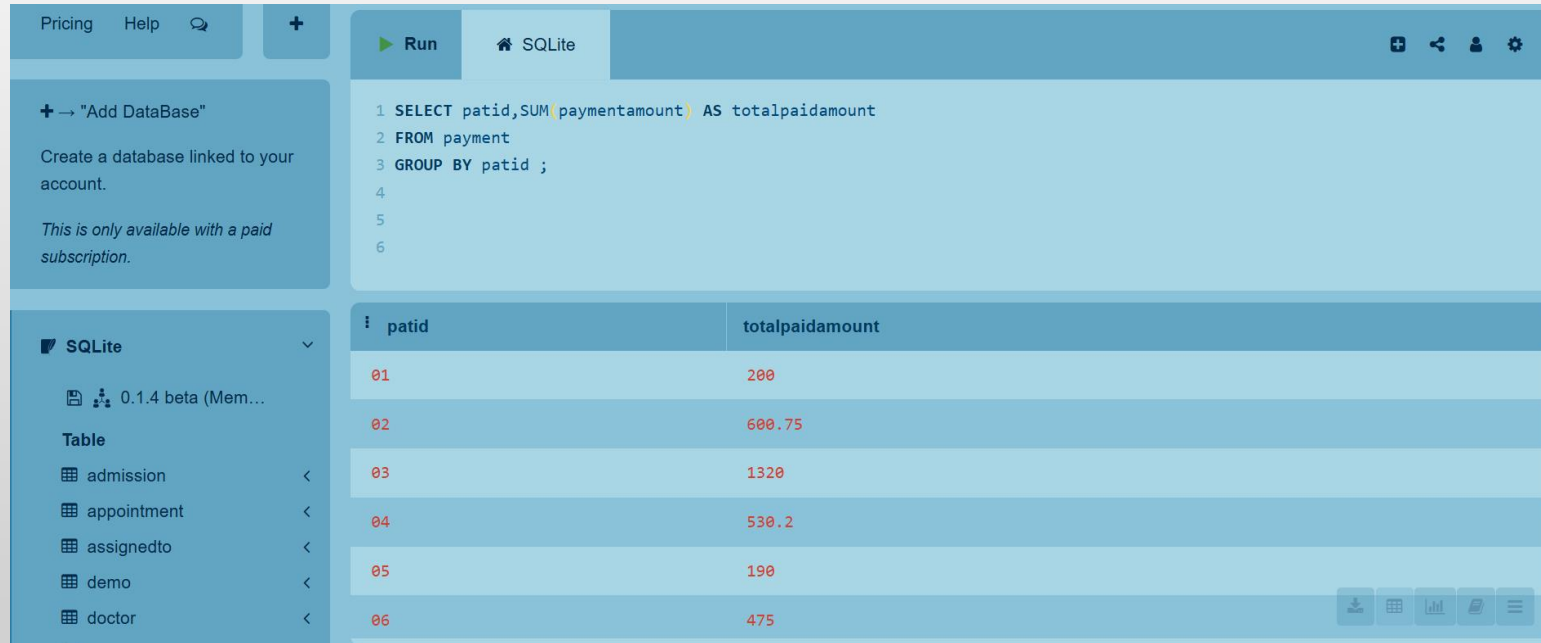
Central Workspace:

- Top bar: "Run" button and "SQLite" tab.
- SQL Editor: A text area containing the following SQL query:

```
1 SELECT docid, COUNT(*)
2 FROM appointment
3 GROUP BY docid;
4
5
6
7
```
- Results Table: A table showing the output of the query.

docid	COUNT(*)
AA	2
AB	3
BB	2
BC	1
- Bottom bar: A row of icons for download, table view, chart view, and other functions.

7. For each patient, retrieve the pat id and the total payment amount for each patient



The screenshot shows a web application interface for SQLite database management. The interface is divided into several sections:

- Top Bar:** Contains "Pricing", "Help", and a "+" icon.
- Left Sidebar:**
 - A section titled "+ → 'Add DataBase'" with instructions: "Create a database linked to your account. This is only available with a paid subscription."
 - A section titled "SQLite" with a version "0.1.4 beta (Mem...)" and a "Table" list: admission, appointment, assignedto, demo, and doctor.
- SQL Editor:** A "Run" button and a "SQLite" tab are visible. The SQL query entered is:

```
1 SELECT patid,SUM(paymentamount) AS totalpaidamount
2 FROM payment
3 GROUP BY patid ;
4
5
6
```
- Results Table:** A table with two columns: "patid" and "totalpaidamount". It displays six rows of data:

patid	totalpaidamount
01	200
02	600.75
03	1320
04	530.2
05	190
06	475
- Bottom Bar:** Contains icons for download, table view, chart view, and other functions.

8. List all the shift day nurses

PricingHelp+

+ → "Add DataBase"

Create a database linked to your account.

This is only available with a paid subscription.

SQLite

Table

admission

appointment

assignedto

demo

doctor

RunSQLite

```
1 SELECT nurseid,nursefirstname,nurselastname,shift
2 FROM nurse
3 WHERE shift = 'Day' ;
4
5
6
```

nurseid	nursefirstname	nurselastname	shift
N1	Daniel	Brown	Day
N4	Sue	Johnson	Day
N5	Tina	Smith	Day

9. For each technician count how many tests are performed



The screenshot displays a web application interface for SQLite database management. The interface is divided into several sections:

- Top Bar:** Contains links for "Pricing", "Help", and a "+" icon.
- Left Sidebar:**
 - A section titled "+ → 'Add DataBase'" with the text "Create a database linked to your account." and a note "This is only available with a paid subscription."
 - A section titled "SQLite" with a dropdown arrow, containing a list of tables: "admission", "appointment", "assignedto", "demo", and "doctor".
- SQL Editor:** A large text area for writing SQL queries. The query entered is:

```
1 SELECT techname, COUNT(*)
2 FROM test
3 GROUP BY techname ;
4
5
6
7
8
```
- Results Table:** A table displaying the results of the SQL query. It has two columns: "techname" and "COUNT(*)". The data rows are:

techname	COUNT(*)
Liam Brooks	1
Sara Finn	3
Tom Hardy	2
- Bottom Bar:** Contains icons for download, table view, chart view, and other functions.

10.Retrieve the room number,availability status and ward name for the room with number '104'

PricingHelp

+

+ → "Add DataBase"
Create a database linked to your account.
This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...
Table
admission
appointment

RunSQLite

```
1 SELECT roomnumber,availabilityst,wardname
2 FROM ward
3 WHERE roomnumber = '104' ;
4
5
6
7
```

roomnumber	availabilityst	wardname
104	Occupied	Ward-4

11. Consider the groups of followupcheck where each group contains the followupchecks that are from the same patients supplied by the same doctor. For each group retrieve the doc id, pat id and number of followupchecks in the group

PricingHelp

+

Create a database linked to your account.
This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

admission

appointment

assignedto

demo

doctor

followupcheck

RunSQLite

```
1 SELECT docid,patid,COUNT(*)
2 FROM followupcheck
3 GROUP BY docid,patid ;
4
```

docid	patid	COUNT(*)
AA	01	1
AA	02	2
BB	03	2
BC	05	1

12. Consider the groups of appointment where each group contains the appointments that are from the same patients supplied by the same doctor. For each group retrieve the doc id, pat id, reason and number of appointments in the group.

The screenshot shows a web application interface for SQLite database management. The interface is divided into three main sections:

- Left Sidebar:** Contains a navigation menu with options like "Pricing", "Help", and a "+" icon. Below this, there is a section titled "Create a database linked to your account." with a note: "This is only available with a paid subscription." At the bottom, there is a list of tables under the heading "SQLite": admission, appointment, assignedto, demo, doctor, followupcheck, and hospital.
- Top Bar:** Includes a "Run" button and a "SQLite" tab. On the right, there are icons for adding a new database, sharing, user profile, and settings.
- Central Area:** Contains a SQL editor with the following query:

```
1 SELECT docid, patid, reason, COUNT(*)
2 FROM appointment
3 GROUP BY docid, patid ;
4
```
- Results Table:** Displays the output of the SQL query. The table has four columns: docid, patid, reason, and COUNT(*). The data is as follows:

docid	patid	reason	COUNT(*)
AA	01	Routine check	1
AA	06	Routine check	1
AB	03	Follow-up	2
AB	05	Routine check	1
BB	04	Routine check	1

13.Retrieve the nurse id and the corresponding ward id.The results are sorted by nurse id in descending order, meaning nurses with higher IDs appear first

PricingHelp+

+ → "Add DataBase"
Create a database linked to your account.
This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...
Table
admission <
appointment <
assignedto <
demo <
doctor <

RunSQLite

```
1 SELECT nurseid, wardid
2 FROM assignedto
3 ORDER BY nurseid DESC ;
4
5
6
```

nurseid	wardid
N5	3
N4	1
N3	2
N2	6
N1	3

DownloadTableChartPrintMenu

14. Consider the groups of followupcheck where each group contains the followupchecks that are from the same patients supplied by the same doctor. For each group that has more than one followupcheck, retrieve the doc id, pat id and nr of followupchecks in the group

PricingHelp+

+ → "Add DataBase"

Create a database linked to your account.

This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

admission

appointment

assignedto

demo

doctor

followupcheck

RunSQLite

```
1 SELECT docid,patid,COUNT(*)
2 FROM followupcheck
3 GROUP BY docid,patid
4 HAVING COUNT(*) > 1 ;
5
6
7
```

docid	patid	COUNT(*)
AA	02	2
BB	03	2

DownloadTableChartExportMenu

15. For each patient having a total payment greater than 450.00, retrieve the patid and the total payment amount for each patient

The screenshot shows a database management interface with a sidebar on the left and a main workspace on the right. The sidebar contains a 'Pricing' and 'Help' menu at the top, followed by a '+ Add DataBase' section with instructions. Below this is a list of databases under the 'SQLite' header, including '0.1.4 beta (Mem...', 'admission', 'appointment', 'assignedto', 'demo', 'doctor', and 'followupcheck'. The main workspace has a 'Run' button and a 'SQLite' tab. The SQL query is entered in the workspace and is as follows:

```
1 SELECT patid,SUM(paymentamount) AS totalpaidamount
2 FROM payment
3 GROUP BY patid
4 HAVING SUM(paymentamount)>= 450.00 ;
5
6
7
```

The results are displayed in a table with two columns: 'patid' and 'totalpaidamount'. The table contains four rows of data:

patid	totalpaidamount
02	600.75
03	1320
04	530.2
06	475

At the bottom right of the workspace, there are icons for downloading, table view, chart view, and other functions.

16. For each payment whose payment amount is below the average payment, retrieve the payment id, payment amount, payment date

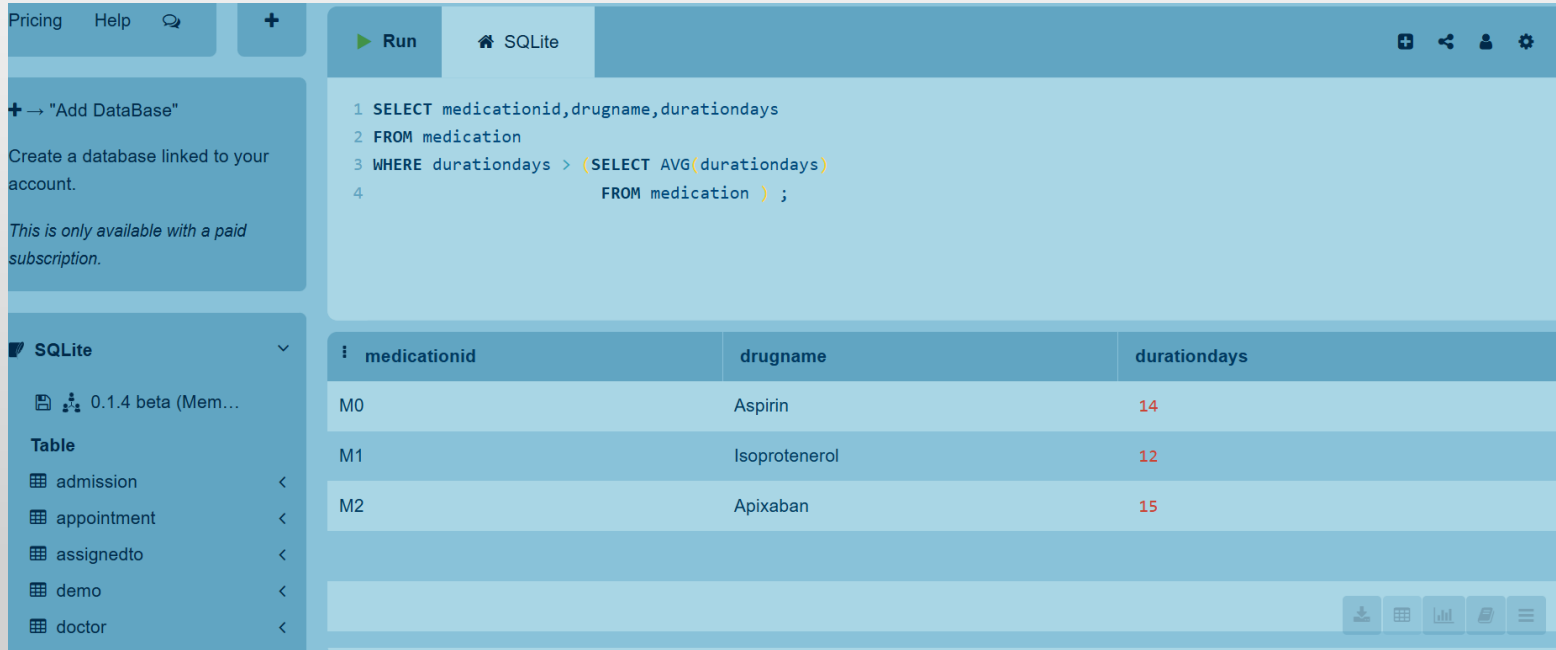


The screenshot shows a SQLite database interface. The top bar includes a 'Run' button and a 'SQLite' tab. The main area displays a SQL query that filters for payments with amounts below the average. The results are shown in a table with three columns: paymentid, paymentamount, and paymentdate. The table contains four rows of data, with the first three rows having their payment IDs (0, 5, 7) and amounts (350.75, 200, 190) highlighted in red.

```
1 SELECT paymentid,paymentamount,paymentdate
2 FROM payment
3 WHERE paymentamount < (SELECT AVG(paymentamount)
4                        FROM payment ) ;
5
6
```

paymentid	paymentamount	paymentdate
0	350.75	2025-03-25
5	200	2024-10-11
7	190	2025-04-14
8	250	2025-03-27

17. For each medication whose duration days is higher than the average duration days, retrieve the medication id, drug name, duration days



Pricing Help

+ → "Add DataBase"

Create a database linked to your account.

This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

- admission
- appointment
- assignedto
- demo
- doctor

Run SQLite

```
1 SELECT medicationid,drugname,durationdays
2 FROM medication
3 WHERE durationdays > (SELECT AVG(durationdays)
4                        FROM medication );
```

medicationid	drugname	durationdays
M0	Aspirin	14
M1	Isoprotenerol	12
M2	Apixaban	15

Download Table Chart Print Menu

18. For each nurse which is assigned to ward with ID 3, retrieve nurse id, nurse first name, nurse last name

PricingHelp+

+ → "Add DataBase"
Create a database linked to your account.
This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

admission

appointment

assignedto

demo

doctor

RunSQLite

```
1 SELECT nurseid, nursefirstname, nurselastname
2 FROM nurse
3 WHERE nurseid IN (
4   SELECT nurseid
5   FROM assignedto
6   WHERE wardid = '3'
7 );
8
```

nurseid	nursefirstname	nurselastname
N1	Daniel	Brown
N5	Tina	Smith

Download

Table

Chart

Print

Menu

19. For each nurse, retrieve the nurse id, nurse first name, nurse last name and procedure name

PricingHelp

+ "Add DataBase"

Create a database linked to your account.
This is only available with a paid subscription.

SQLite

0.1.4 beta (Mem...)

Table

admission

appointment

assignedto

demo

doctor

followupcheck

RunSQLite

```
1 SELECT n.nurseid,n.nursefirstname,n.nurselastname,p.proceduretype
2 FROM nurse n, procedure p
3 WHERE n.procedureid = p.procedureid ;
4
```

nurseid	nursefirstname	nurselastname	proceduretype
N1	Daniel	Brown	Echocardiography
N2	Anna	Swift	Holter Monitoring
N3	Eva	Clark	Cardioversion
N4	Sue	Johnson	Coronary Angioplasty
N5	Tina	Smith	Pacemaker insertion

Download

Table

Chart

Print

Menu

20. For each procedure, retrieve the procedure id, procedure date, procedure type, patient name, patient last name

The screenshot shows a database management interface with a sidebar on the left containing a list of database tables: appointment, assignedto, demo, doctor, followupcheck, hospital, medication, nurse, patient, payment, procedure, test, and ward. Below these are two database connections: MariaDB and PostgreSQL. The main area is titled 'Run' and 'SQLite'. It contains a SQL query:

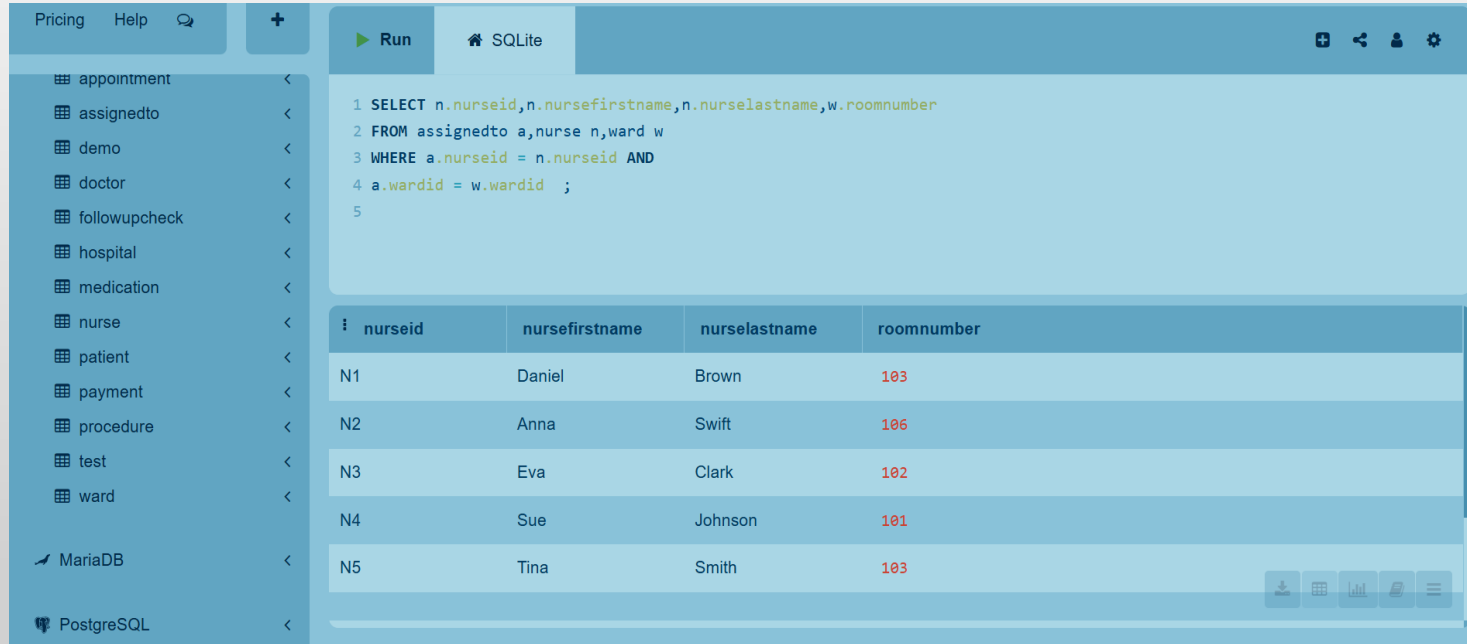
```
1 SELECT p.procedureid,p.proceduredt,p.proceduretype,pt.patfirstname,pt.patlastname
2 FROM procedure p,patient pt
3 WHERE p.patid = pt.patid ;
4
```

Below the query, the results are displayed in a table with the following columns: procedureid, proceduredt, proceduretype, patfirstname, and patlastname. The table contains six rows of data:

procedureid	proceduredt	proceduretype	patfirstname	patlastname
P1	2024-11-20	Coronary Angioplasty	Tom	Wilson
P2	2024-10-05	Pacemaker insertion	Emily	Johnson
P3	2025-03-11	Echocardiography	John	Williams
P4	2024-07-15	Cardioversion	Jane	White
P5	2025-04-27	Cardiac Catheterization	Jackson	Roberts
P6	2025-02-08	Holter Monitoring	Olivia	Scott

At the bottom right of the results table, there are icons for downloading, grid view, chart view, print, and a menu.

21. For each nurse assignment, retrieve the nurse ID, first name, last name, and the room number of the ward they are assigned to

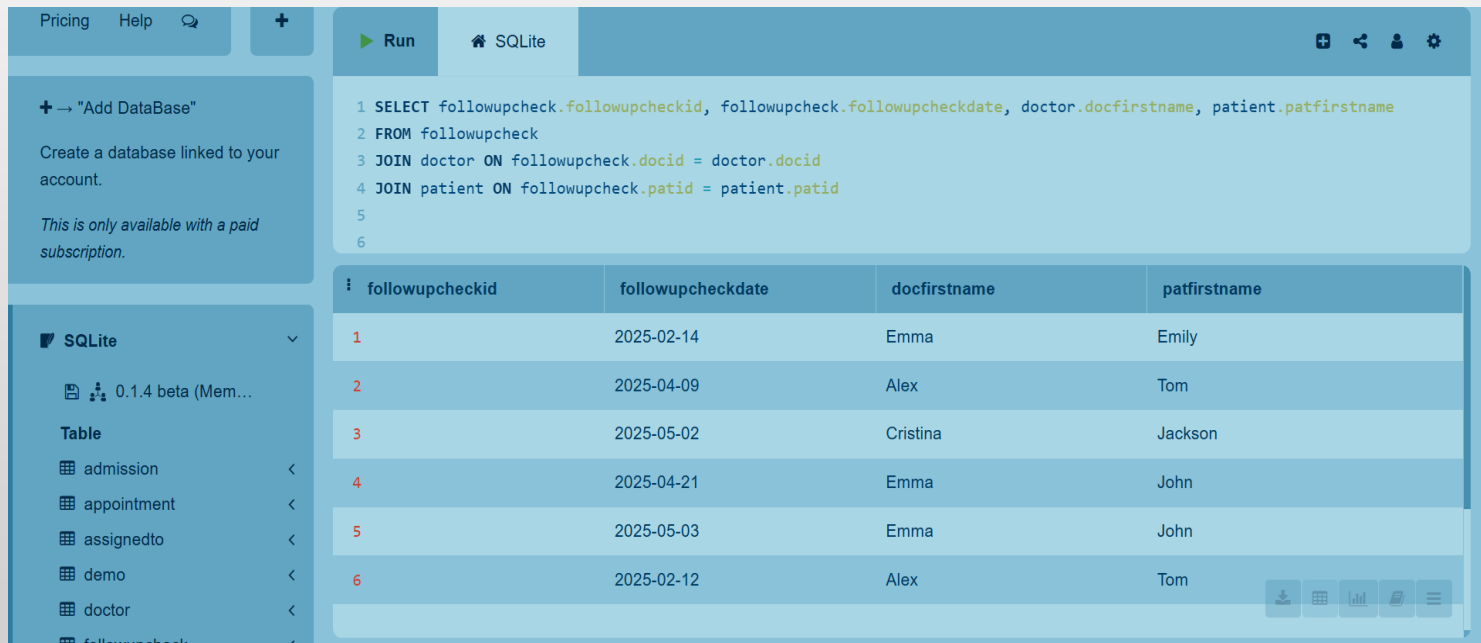


The screenshot shows a database management interface with a sidebar on the left containing a list of database tables: appointment, assignedto, demo, doctor, followupcheck, hospital, medication, nurse, patient, payment, procedure, test, and ward. Below these is a section for database engines: MariaDB and PostgreSQL. The main area is titled 'Run' and 'SQLite'. It contains an SQL query that selects nurse ID, first name, last name, and room number from the 'assignedto' table, joined with 'nurse' and 'ward' tables. The results are displayed in a table with 5 rows.

```
1 SELECT n.nurseid,n.nursefirstname,n.nurselastname,w.roomnumber
2 FROM assignedto a,nurse n,ward w
3 WHERE a.nurseid = n.nurseid AND
4 a.wardid = w.wardid ;
5
```

nurseid	nursefirstname	nurselastname	roomnumber
N1	Daniel	Brown	103
N2	Anna	Swift	106
N3	Eva	Clark	102
N4	Sue	Johnson	101
N5	Tina	Smith	103

22. List all follow-up checks with doctor and patient names

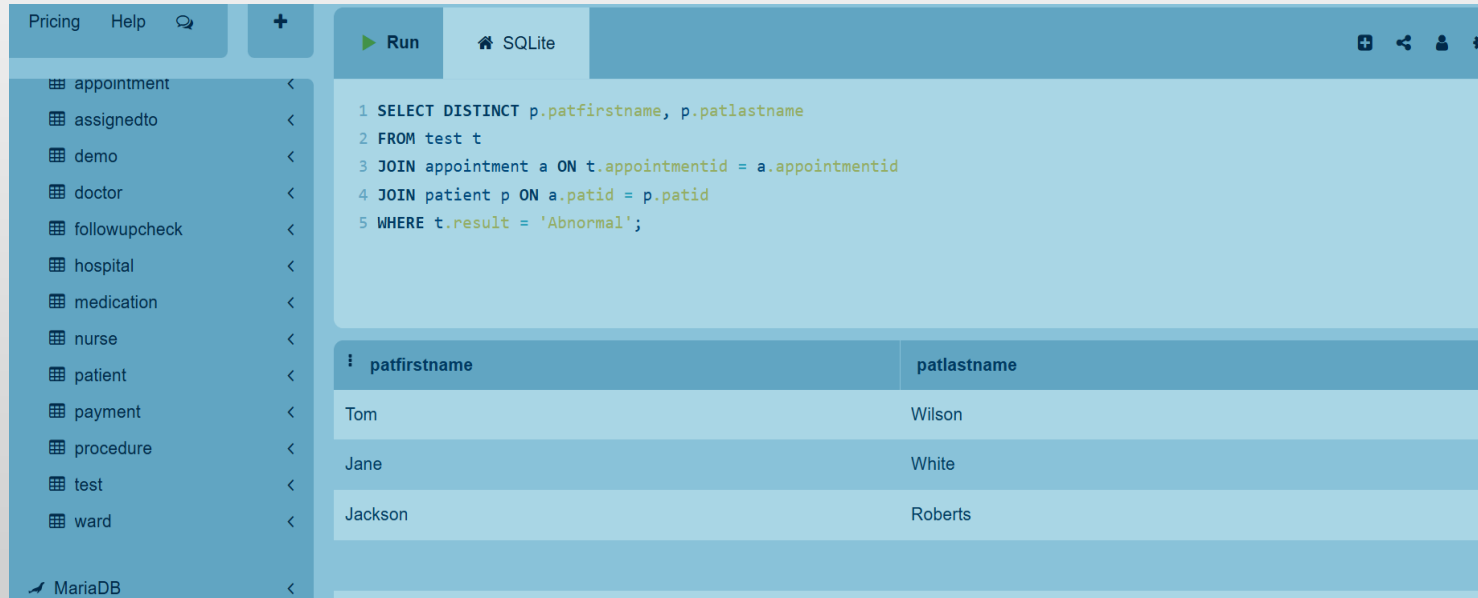


The screenshot shows a database management interface. On the left, there's a sidebar with a 'Pricing' link, a 'Help' link, and a '+ Add DataBase' button. Below this, it says 'Create a database linked to your account. This is only available with a paid subscription.' Further down, there's a section for 'SQLite' with a dropdown arrow. Under 'Table', there's a list of tables: 'admission', 'appointment', 'assignedto', 'demo', 'doctor', and 'followupcheck'. The main area has a 'Run' button and a 'SQLite' tab. The SQL query is displayed in a light blue box, and the results are shown in a table below it. The table has four columns: 'followupcheckid', 'followupcheckdate', 'docfirstname', and 'patfirstname'. There are six rows of data, each with a red number in the first column. At the bottom right of the table, there are icons for download, grid, chart, and other functions.

```
1 SELECT followupcheck.followupcheckid, followupcheck.followupcheckdate, doctor.docfirstname, patient.patfirstname
2 FROM followupcheck
3 JOIN doctor ON followupcheck.docid = doctor.docid
4 JOIN patient ON followupcheck.patid = patient.patid
5
6
```

followupcheckid	followupcheckdate	docfirstname	patfirstname
1	2025-02-14	Emma	Emily
2	2025-04-09	Alex	Tom
3	2025-05-02	Cristina	Jackson
4	2025-04-21	Emma	John
5	2025-05-03	Emma	John
6	2025-02-12	Alex	Tom

23.Retrieve the patient first name,patient last name for the patients who had a test with abnormal result



The screenshot shows a database management interface with a sidebar on the left containing a list of database tables: appointment, assignedto, demo, doctor, followupcheck, hospital, medication, nurse, patient, payment, procedure, test, and ward. The main area is divided into two sections. The top section contains an SQL query:

```
1 SELECT DISTINCT p.patfirstname, p.patlastname
2 FROM test t
3 JOIN appointment a ON t.appointmentid = a.appointmentid
4 JOIN patient p ON a.patid = p.patid
5 WHERE t.result = 'Abnormal';
```

 The bottom section displays the results of the query in a table format with two columns: patfirstname and patlastname. The results are as follows:

patfirstname	patlastname
Tom	Wilson
Jane	White
Jackson	Roberts

24.Retrieve the doc first name and last name for doctors who performed procedures in 2025

The screenshot shows a database management interface with a sidebar on the left and a main workspace on the right. The sidebar contains a 'Pricing' link, a 'Help' link, and a '+ Add DataBase' button. Below this, it says 'Create a database linked to your account. This is only available with a paid subscription.' The main workspace has a 'Run' button and a 'SQLite' tab. The SQL query is displayed in the main area, and the results are shown in a table below it.

SQL Query:

```
1 SELECT DISTINCT d.docfirstname ,d.doclastname
2 FROM doctor d
3 JOIN procedure p ON p.docid = d.docid
4 WHERE p.proceduredt >= '2025-01-01' ;
5
6
```

Results Table:

docfirstname	doclastname
Alex	James
Gregory	Bailey
Emma	Yang

25. Find all the appointment ids and appointment reasons for patient 'Olivia'

PricingHelp

+

RunSQLite

appointment

assignedto

demo

doctor

followupcheck

hospital

medication

nurse

patient

payment

procedure

test

```
1 SELECT a.appointmentid,a.reason
2 FROM appointment a
3 JOIN patient p ON a.patid = p.patid
4 WHERE p.patfirstname = 'Olivia' ;
```

appointmentid	reason
A110	Initial consult
A111	Routine check

THANK YOU FOR YOUR TIME!

WORKED:Siera Kurpali

